



COMP1006 - WEEK SIX

**UPDATE & DELETE // MIDTERM
PREP**

TODAY'S SCHEDULE

1. Check In
2. Review Create & Read
3. Adding Update & Delete
4. Midterm Preparation
5. Course Project Phase One Work Period
6. Summary, Task List, Next Week

```
JS get_users_payload.js  JS get_tweets_payload.py  JS get-user.js  JS get-tweet.js
Users > aspecker > Desktop > scripts > curl > JS get-user.js > @accessToken
16 const accessTokenURL = new URL('https://api.twitter.com/oauth/access_token');
17 const authorizeURL = new URL('https://api.twitter.com/oauth/authorize');
18 const endpointURL = new URL('https://api.twitter.com/labs/1/users');
19
20 const params = {
21   usernames: 'AureliaSpecker',
22   format: 'detailed'
23 };
24
25 async function input(prompt) {
26   return new Promise(async (resolve, reject) => {
27     readline.question(prompt, (out) => {
28       readline.close();
29       resolve(out);
30     });
31   });
32 }
33
34 async function accessToken(oauth_token, oauth_token_secret, verifier) {
35   const oAuthConfig = {
36     consumer_key: ConsumerKey,
37     consumer_secret: ConsumerSecret,
38     token: oauth_token,
39     token_secret: oauth_token_secret,
40     verifier: verifier,
41   };
42
43   const req = await post({url: accessTokenURL, oauth: oAuthConfig});
44   if (req.body) {
45     return qs.parse(req.body);
46   } else {
47     throw new Error('Cannot get an OAuth request token');
48   }
49 }
50
51 async function requestToken() {
52   const oAuthConfig = {
53     callback: 'oob',
54     consumer_key: ConsumerKey,
55     consumer_secret: ConsumerSecret,
56   };
57
58   const req = await post({url: requestTokenURL, oauth: oAuthConfig});
59   if (req.body) {
60     return qs.parse(req.body);
61   } else {
```


LEARNING OBJECTIVES

1. Review Create & Read functionality from last week
2. Add update & delete functionality to our web application
3. Prepare for your Midterm Exam (20%) - next week IN CLASS



```
JS get_users_payload.js  get_tweets_payload.py  JS get-user.js  JS get-tweet.js
Users > aspecker > Desktop > scripts > URL > JS get-user.js @accessToken
16 const accessTokenURL = new URL('https://api.twitter.com/oauth/access_token');
17 const authorizeURL = new URL('https://api.twitter.com/oauth/authorize');
18 const endpointURL = new URL('https://api.twitter.com/labs/1/users');
19
20 const params = {
21   usernames: 'AureliaSpecker',
22   format: 'detailed'
23 };
24
25 async function input(prompt) {
26   return new Promise(async (resolve, reject) => {
27     readline.question(prompt, (out) => {
28       readline.close();
29       resolve(out);
30     });
31   });
32 }
33
34 async function accessToken(oauth_token, oauth_token_secret, verifier) {
35   const OAuthConfig = {
36     consumer_key: ConsumerKey,
37     consumer_secret: ConsumerSecret,
38     token: oauth_token,
39     token_secret: oauth_token_secret,
40     verifier: verifier,
41   };
42   const req = await post({url: accessTokenURL, oauth: OAuthConfig});
43   if (req.body) {
44     return qs.parse(req.body);
45   } else {
46     throw new Error('Cannot get an OAuth request token');
47   }
48 }
49
50
51 async function requestToken() {
52   const OAuthConfig = {
53     callback: 'oob',
54     consumer_key: ConsumerKey,
55     consumer_secret: ConsumerSecret,
56   };
57   const req = await post({url: requestTokenURL, oauth: OAuthConfig});
58   if (req.body) {
59     return qs.parse(req.body);
60   } else {
61     throw new Error('Cannot get an OAuth request token');
62   }
63 }
```

COMP1006 - WEEK SIX

WEEK 6 - CHECK IN

Week Five – REVIEW

Better Way to Track Our Orders?



Lisa Dough <lisa@bakeittillyoumakeit.com>

Today, 10:17 AM

Hi Team,

As you know, it's our busy season and we're getting lots of customer orders coming through via email.

I was wondering if you could help me find a better way to store and track our orders? 👍

Trying to keep track of everything this way is getting overwhelming.

Do you think there's a more organized solution? 😊

Thanks again, Lisa

Lisa Dough

[Bake It Till You Make It Bakery](#) 🏠

↩ Reply

➡ Forward

HOW WEB APPS TALK TO THE DATABASE

Typical Workflow:

- ▶ User submits a form
- ▶ PHP receives the user input
- ▶ PHP builds an SQL query
- ▶ Database executes the query
- ▶ Data is stored or returned

**** if handled incorrectly, user input can change how SQL behaves**

CREATE/INSERT

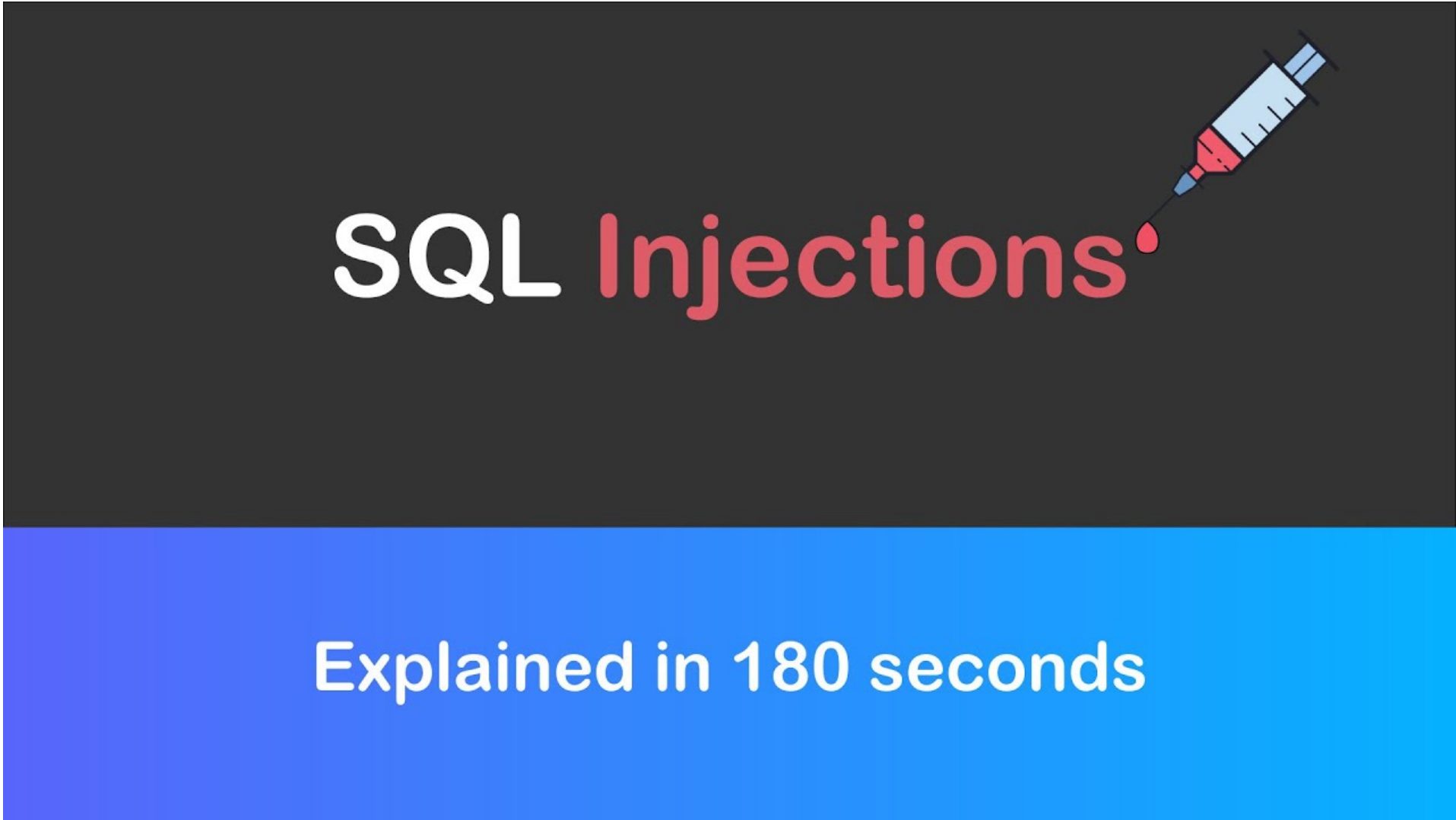
- ▶ Once you **connect to a database**, you can use the connection to execute **SELECT** statements that retrieve data and **INSERT UPDATE** and **DELETE**
- ▶ can use **prepared statements** or dynamic statements

DYNAMIC STATEMENT – WHAT COULD GO WRONG?

php

```
$sql = "SELECT * FROM users WHERE email = '$email'";
```

WHAT IS AN SQL INJECTION?



SQL Injections

Explained in 180 seconds

ISN'T SANITIZATION ENOUGH?



USE PREPARED STATEMENTS

- ▶ Separate **SQL structure** from **data**
- ▶ Prevent **user input from changing query logic**
- ▶ Are built into **PDO**
- ▶ Works by sending SQL structure to the database and user data is sent separately
- ▶ User input is **treated as data only**
- ▶ Prepared statements are the **industry standard**

HOW TO USE PREPARED STATEMENTS

1. Write SQL with **placeholders**
2. **Prepare** the statement
3. **Execute** with **values**

PLACEHOLDERS

```
INSERT INTO songs (title, genre)
VALUES (:title, :genre)
```

- ▶ Act like **variables** in SQL
- ▶ Are **replaced safely** at **execution time**

PREPARE THE STATEMENT

```
$stmt = $pdo->prepare($sql);
```

- ▶ Sends **SQL structure** to the database
- ▶ Database **parses** and **validates** the query
- ▶ No user data is sent **yet**

EXECUTE

```
$stmt->execute([  
    ":title" => $title,  
    ":genre" => $genre  
]);
```

- ▶ Sends user data to the database
- ▶ Safely inserts values into the placeholders
- ▶ Runs the SQL query

LET'S BUILD THIS TOGETHER

UPDATE & DELETE

MIDTERM EXAM INFO

MIDTERM EXAM INFORMATION

- ▶ Held **IN CLASS** next week. **You MUST attend class in order to complete your midterm**
- ▶ Midterm is worth 20% of your final grade
- ▶ Includes all material/topics covered in Weeks 1 - 6

MIDTERM EXAM INFORMATION

- ▶ **Open book** - you are able to use all lecture PDFs, code files and resources included in the Week 1 - 6 folders, as well as your own notes
- ▶ You are **NOT** permitted to use Google / Stackoverflow / any LLM etc.
- ▶ All answers should **represent your own thoughts/work**
- ▶ Your midterm will include T/F, Multiple Choice, as well as one or two short answer questions/application questions in which you will be asked to explain code or write your own code

TO DO

COMP 1006 - Winter 2026

LAB FOUR – CREATE & READ

To Complete This Lab:

A client has asked you to help them set up a simple email subscription feature for their website so they can start building a mailing list. They already have a basic subscription form, but they need the backend functionality to store subscriber information in a database and view a list of subscribers. You have been given starter code that includes TODO comments to guide you through the process. Your task is to complete the missing code using PDO prepared statements to insert subscriber data into the database and retrieve it for display.

Learning Objectives:

- design, program, and implement Web pages that interact with Web-enabled databases performing simple CRUD (Create, Read, Update, Delete) operations;

TO DO THIS WEEK :

LAB FOUR:

SECTION 01 – FRIDAY, FEB 13TH @ 11:59 PM

SECTION 02 – SUNDAY, FEB 15TH @ 11:59 PM

CONTINUE WORKING ON COURSE PROJECT PHASE ONE –

******* NEW DUE DATE *******

DUE WEEK 7

A FEW TIPS...

- ▶ Don't leave it to the last minute
- ▶ Don't be tempted to use someone's code!
- ▶ Use this as an opportunity to apply what you have learned so far this semester
- ▶ Do not use mysqli_ (we discussed and implemented the PDO extension)
- ▶ Do ask questions/ seek help if you need it!

SUMMARY, TO DO & NEXT WEEK

SUMMARY

1. Reviewed Create & Read
2. Added Update & Delete Functionality
3. Prepared for Midterm Exam
4. Reviewed Course Project Phase One
5. Introduced Lab Four



```
JS get_users_payload.js  get_tweets_payload.py  JS get-user.js  JS get-tweet.js
Users > aspecker > Desktop > scripts > curl > JS get-user.js @accessToken
16 const accessTokenURL = new URL('https://api.twitter.com/oauth/access_token');
17 const authorizeURL = new URL('https://api.twitter.com/oauth/authorize');
18 const endpointURL = new URL('https://api.twitter.com/labs/1/users');
19
20 const params = {
21   usernames: 'AureliaSpecker',
22   format: 'detailed'
23 };
24
25 async function input(prompt) {
26   return new Promise(async (resolve, reject) => {
27     readline.question(prompt, (out) => {
28       readline.close();
29       resolve(out);
30     });
31   });
32 }
33
34 async function accessToken(oauth_token, oauth_token_secret, verifier) {
35   const oAuthConfig = {
36     consumer_key: ConsumerKey,
37     consumer_secret: ConsumerSecret,
38     token: oauth_token,
39     token_secret: oauth_token_secret,
40     verifier: verifier,
41   };
42
43   const req = await post({url: accessTokenURL, oauth: oAuthConfig});
44   if (req.body) {
45     return qs.parse(req.body);
46   } else {
47     throw new Error('Cannot get an OAuth request token');
48   }
49 }
50
51 async function requestToken() {
52   const oAuthConfig = {
53     callback: 'oob',
54     consumer_key: ConsumerKey,
55     consumer_secret: ConsumerSecret,
56   };
57
58   const req = await post({url: requestTokenURL, oauth: oAuthConfig});
59   if (req.body) {
60     return qs.parse(req.body);
61   } else {
62     throw new Error('Cannot get an OAuth request token');
63   }
64 }
```


NEXT WEEK :
MIDTERM EXAM!